FeBEx: In-Network Deduplication for LoRaWAN Backhaul Using P4 Programmable Switches

ANONYMOUS AUTHOR(S)

LoRaWAN and Helium networks solve the last-mile connectivity problem for IoT devices, but their fundamental architecture creates a critical inefficiency: when a device broadcasts, multiple geographically distributed gateways independently forward the same packet through backhaul to cloud backends. This redundancy, essential for reliability, multiplies backhaul traffic and cost.

We present FeBEx, a P4-programmable aggregation switch that moves deduplication from cloud backends to the network edge. FeBEx implements register-based deduplication with epoch-driven expiry, integrated with Longest-Prefix-Match tenant steering and packet cloning for Proof-of-Coverage attribution. Through comprehensive evaluation on Mininet-BMv2, we demonstrate that FeBEx achieves 50-90% backhaul bandwidth savings while maintaining perfect delivery ratio (no false suppressions), 100% multi-tenant isolation, and accurate payment attribution. We provide practical guidelines: register size $\geq 4N$, epoch intervals of 5-10 seconds, and validate effectiveness across city-scale deployments (50-500 devices, 5-50 gateways). This work demonstrates that edge-based deduplication is viable, effective, and ready for deployment on production P4 switches.

1 Introduction

Helium and LoRaWAN networks have emerged as key infrastructure for low-power, wide-area connectivity in IoT deployments. However, the fundamental architecture of these networks creates a significant inefficiency: when a device broadcasts a single LoRa transmission, multiple geographically distributed hotspots may receive and independently forward the same packet to the cloud backend. This redundancy, while essential for reliability and network coverage, wastes precious backhaul bandwidth — a scarce and costly resource in network infrastructure.

Consider a typical Helium network deployment where hotspots are distributed across a city. When an IoT device transmits, perhaps five different hotspots receive the signal. Each hotspot independently sends a complete UDP/IP copy of the packet through the backhaul to a cloud data center. The cloud then performs packet deduplication in software, discarding the redundant copies. This results in a 5× multiplication of backhaul traffic for a single logical uplink, directly translating to increased bandwidth costs, higher latency, and reduced effective network capacity.

The conventional approach handles this duplication at the cloud tier — a software post-processing step after packets have already consumed backhaul resources. FeBEx (Filtering in-network for Backhaul experiments) proposes a fundamental architectural shift: move deduplication into the network itself, specifically into a P4-programmable switch positioned as an aggregation point between hotspots and cloud backends.

1.1 Motivation and Problem Statement

The key insight is that deduplication can be solved efficiently in-network, at the edge where the switch can observe all incoming copies simultaneously. By implementing deduplication as a stateful forwarding action in a P4 programmable data plane, FeBEx achieves three critical benefits:

(1) **Backhaul bandwidth savings**: Duplicate packets are suppressed before egress, reducing backhaul traffic by 50-90% depending on the degree of duplication.
(2) **Latency reduction**: The first copy reaches the backend immediately, without waiting for software-based cloud deduplication.
(3) **Payment accuracy**: A P4 clone session mirrors the first-forwarded packet to a separate Helium Cloud endpoint, enabling precise Proof-of-Coverage payments to the hotspot that legitimately delivered the uplink.

1.2 Research Questions

This work addresses the following research questions:

(1) **Correctness**: Can an in-network deduplication filter correctly suppress true duplicates without falsely suppressing unique packets, even under high-collision hash conditions?
(2) **Efficiency**: How much backhaul bandwidth can in-network deduplication realistically save, and how does this scale with network topology and traffic patterns?
(3) **State requirements**: How large must the deduplication state (register arrays) be to achieve acceptable loss rates across different deployment scales?
(4) **Temporal sensitivity**: How sensitive is the deduplication mechanism to epoch rotation frequency, and is there a sweet spot that minimizes boundary leakage?
(5) **Scalability**: Does the approach maintain effectiveness across city-scale deployments with hundreds of devices and dozens of hotspots?
(6) **Multi-tenancy**: Can the P4 pipeline correctly handle multiple independent tenant networks through LPM-based traffic steering without cross-contamination?

1.3 Contributions

This work presents:

(1) A complete P4-16 implementation of an in-network deduplication pipeline for LoRaWAN/Helium networks, running on BMv2 (v1model) with P4Runtime control plane.
(2) An evaluation framework (Mininet-based) that systematically explores parameter space: duplicate factors, register sizes, epoch intervals, network scales, and multi-tenant configurations.
(3) Seven comprehensive experiments (E1-E7) that validate correctness, measure realistic backhaul savings, and identify practical design parameters (e.g., register sizing, epoch interval tuning).
(4) An interactive visualization dashboard that animates the deduplication process in real-time across four city-scale scenarios.
(5) Empirical evidence that register-based deduplication with simple hash collision handling is viable for production-scale IoT network aggregation.

1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 surveys related work in network deduplication, P4 programming, and IoT backhaul optimization. Section 3 provides necessary background on LoRaWAN, Helium networks, and P4 programmable switches. Section 4 describes the FeBEx system architecture, including the P4 data plane, the control plane, and the evaluation topology. Section 5 details the implementation, including packet format, P4 pipeline structure, and controller logic. Section 6 describes our experimental methodology and the seven experiments. Section 7 presents empirical results and analysis for each experiment. Section 8 discusses implications, limitations, and future work. Finally, Section 9 concludes.

2 Related Works

This work intersects several research areas: network deduplication, P4 programmable switching, IoT backhaul optimization, and in-network processing.

2.1 Network Deduplication

Packet deduplication at the network layer is well-studied. Traditional approaches operate at the cloud or core network layer. *Cloud-based deduplication* is the standard in Helium and similar networks, where a centralized backend receives all copies and performs software-based duplicate detection using in-memory lookup tables or bloom filters. While simple and flexible, this approach wastes upstream bandwidth and processing resources.

Intermediate deduplication has been explored in various contexts. For example, in wireless sensor networks, in-network packet aggregation and duplicate suppression have been used to reduce energy consumption [9]. However, these approaches typically operate on lower-layer protocols (e.g., at the MAC layer) and do not address the specific challenges of heterogeneous IoT backhaul networks.

Middle-box deduplication solutions (e.g., load balancers, NAT boxes) can perform stateful deduplication, but require dedicated middlebox infrastructure and do not integrate into the forwarding datapath. FeBEx differs by embedding deduplication as a first-class forwarding action in a P4-programmable switch, eliminating separate infrastructure.

2.2 P4 and Programmable Networks

P4 [3] is a domain-specific language for programming network packet forwarding in data plane devices. Since its introduction in 2014, P4 has enabled researchers and operators to implement custom network protocols and services without modifying underlying hardware or relying on vendor-specific instruction sets.

Notable P4-based services include:

(1) **Stateful packet processing**: Services like INT (In-band Network Telemetry) [7] use P4 registers and arithmetic to collect per-packet metrics. FeBEx applies similar register-based state to track deduplication keys.
(2) **Load balancing**: P4-based load balancers [1, 5] use hash functions and port selection logic implemented in P4 pipelines. FeBEx similarly uses dual CRC32 hashing for key indexing and verification.
(3) **In-network aggregation**: Techniques like data aggregation in switches have been explored to reduce bandwidth in data center and wide-area networks [6].
(4) **Multi-tenant isolation**: P4 pipelines can enforce tenant isolation through LPM-based forwarding and table-entry-based access control, as demonstrated in virtual network slicing systems.

Our work applies P4 to a new domain (IoT backhaul deduplication) and validates practical design choices (register sizing, hash collision handling, epoch-based expiry) that may be applicable to other stateful P4 services.

2.3 Hash-Based Deduplication

Bloom filters and hash tables are classical approaches to deduplication. A naive approach stores the full packet or a cryptographic hash and performs exact-match lookup. However, this is expensive in switch memory.

FeBEx uses a two-level hash approach: a primary hash function selects the register index, and a second hash function serves as a verification key to disambiguate collisions. This is a well-known technique in hash table design [4] and is adopted here to fit within typical P4 register constraints (typically 32-64 KB per switch).

Bloom filters offer probabilistic deduplication with tunable false-positive rates [2]. FeBEx does not use Bloom filters directly; instead, it implements a deterministic hash table with overflow handling through register size tuning. This trades off some flexibility for simplicity and determinism in a P4 context.

2.4 IoT and LoRaWAN Infrastructure

LoRaWAN is a Low Power Wide Area Network (LPWAN) protocol standardized by the LoRa Alliance. Helium is a community-driven network operating at LoRaWAN frequencies.

Infrastructure challenges in LoRaWAN include:

(1) **Backhaul congestion**: As networks scale, the backhaul from gateways to cloud becomes a bottleneck, especially in dense urban deployments.
(2) **Packet loss**: Congestion and limited gateway capacity lead to packet loss, reducing network reliability.
(3) **Cost**: Backhaul bandwidth is expensive, especially for deployment in cost-sensitive regions.
(4) **Proof-of-Coverage**: In incentivized networks like Helium, gateways must be compensated based on coverage contribution. This requires accurate attribution of which gateway delivered a given packet.

FeBEx directly addresses backhaul congestion and cost by moving deduplication from cloud to edge. It maintains Proof-of-Coverage accuracy through clone sessions that mirror authenticated receipts to the Helium Cloud.

2.5 Edge Computing and Programmable Switches

The emergence of programmable switches (Barefoot Tofino, NetronomeAgilio, etc.) has enabled a new class of *edge services* — stateful services running in the forwarding datapath rather than in separate appliances or cloud servers. Examples include:

(1) **Network monitoring and telemetry**: Switches can collect and aggregate statistics in-band [7].
(2) **Consensus and coordination**: In-switch services have been used to implement distributed state machine replication [8].
(3) **Load balancing**: Switches can make intelligent forwarding decisions based on live server state [1].

FeBEx is positioned in this space as an edge deduplication service. By implementing deduplication in the switch, we move a CPU-bound cloud operation into hardware-accelerated forwarding, reducing latency and cost.

2.6 Summary and Positioning

FeBEx is unique in combining:

(1) **Domain focus**: Specific to IoT/LoRaWAN backhaul (not general-purpose deduplication).
(2) **Integration**: Embedded in P4 forwarding pipeline with stateful registers (not a separate middlebox).
(3) **Evaluation**: Comprehensive empirical validation on realistic LoRaWAN topologies and traffic patterns.
(4) **Proof-of-Coverage awareness**: Maintains attribution accuracy through clone sessions, critical for incentivized networks.

We distinguish this work from prior cloud-based deduplication by showing that edge-based deduplication is both feasible and beneficial, and from general P4 stateful processing literature by providing concrete design parameters and tradeoffs for IoT backhaul specifically.

3 Background

This section provides necessary background on LoRaWAN networks, the Helium ecosystem, and P4 programmable switches.

3.1 LoRaWAN and Helium Networks

3.1.1 LoRaWAN Overview. LoRaWAN is a Low Power Wide Area (LPWA) networking protocol designed for battery-operated devices over long distances (10-20+ km in rural areas). It operates in unlicensed ISM bands (e.g., 915 MHz in North America, 868 MHz in Europe) and is standardized by the LoRa Alliance.

A typical LoRaWAN network consists of:

(1) **End devices (EDs)**: Battery-powered IoT sensors that transmit data at very low power, sacrificing bandwidth for range and energy efficiency.
(2) **Gateways (hotspots)**: Receive LoRa signals from EDs and forward them to a Network Server (LNS) via IP backhaul.
(3) **Network Server**: Processes uplinks (ED to cloud) and downlinks (cloud to ED), performs deduplication, and routes to application backends.

Key characteristics:

- *Broadcast nature*: A single ED transmission is received by *all* gateways in range (typically 3-10 gateways in urban areas, 1-3 in rural areas).
- *Redundancy by design*: Multiple copies per uplink ensure reliability; loss of a single gateway doesn't break the link.
- *Backhaul inefficiency*: This redundancy must traverse the backhaul network. In a city with 100 gateways and 1,000 active devices, a typical device might be heard by 4 gateways, generating 4,000 redundant backhaul packets per 1,000 logical uplinks.

3.1.2 Helium Network. Helium is a community-run network that incentivizes hotspot operators to deploy gateways by rewarding them with HNT cryptocurrency. Rewards are proportional to *Proof-of-Coverage (PoC)* — a mechanism by which a gateway's genuine coverage contribution is verified.

In Helium's PoC model, a gateway earns rewards based on:

(1) *Data transfer*: Rewarded per packet successfully forwarded to the LNS.
(2) *Witness attestation*: Other gateways witness and attest to a gateway's coverage beacon, confirming its geographic presence.

Accurate attribution is critical: if the PoC mechanism cannot determine which gateway originally heard a packet, reward distribution becomes ambiguous. FeBEx preserves this attribution through clone sessions (see Section 4).

3.2 P4 Programmable Switches

3.2.1 P4 Language and Execution Model. P4 (Programming Protocol-independent Packet Processing) is a domain-specific language that allows programmers to define custom packet processing logic that runs in switch data planes. A P4 program specifies:

(1) **Headers**: Custom packet headers and metadata.
(2) **Parser**: State machine that extracts headers from raw packets.
(3) **Ingress pipeline**: Match-action tables and stateful operations on matched packets.
(4) **Egress pipeline**: Final packet modification and forwarding decisions.
(5) **Deparser**: Reconstructs the packet for transmission.

Execution follows a strict deterministic model: each packet is processed by the parser, then independently by the ingress pipeline, egress pipeline, and deparser. Modern P4 switches support both stateless operations (standard forwarding, header rewriting) and stateful operations (register reads/writes, packet mirroring, recirculation).

3.2.2 BMv2 and v1model. BMv2 (Behavioral Model v2) is a software switch implementation used for development and testing. It implements the *v1model* architecture, which is one of several P4 target architectures.

v1model provides:

(1) **Packet I/O**: Standard Linux interfaces (TAP devices or pcap files).
(2) **Match-action tables**: Single or cascaded LPM, exact match, ternary match.
(3) **Registers**: Arrays indexed by packet-derived values, supporting atomic read-modify-write.
(4) **Counters**: Automatic per-entry or per-packet statistics.
(5) **Cloning**: I2E (ingress-to-egress) and E2E (egress-to-egress) cloning for packet replication.
(6) **Recirculation**: A packet can be fed back to the ingress pipeline for multi-pass processing.

3.2.3 P4Runtime and Control Plane. P4Runtime is a standardized API for controlling a P4 switch at runtime. It allows a control plane program (e.g., a Python controller) to:

(1) Install and remove forwarding table entries.
(2) Read and write packet counters and registers.
(3) Configure cloning sessions and other switch parameters.
(4) Receive notifications (e.g., when a switch becomes active).

Communication occurs over gRPC, a high-performance RPC framework. In our implementation, we use *finsy*, a Python library that provides a Pythonic interface to P4Runtime.

3.3 Deduplication Fundamentals

3.3.1 Hash-Based Lookup. A deduplication system must maintain a data structure that answers the query: *"Have I seen this packet before?"* The most efficient approach in constrained hardware is hash-based lookup:

(1) **Hash function**: Map a packet's *identifying features* (e.g., DevAddr + FCnt) to an index h in a register array.
(2) **Verification key**: Store a secondary hash (the *key*) in the register to disambiguate collisions.
(3) **Epoch**: Store a timestamp (epoch) to invalidate old entries.

Formally, the lookup algorithm is:

(1) Compute $h = \text{hash_index}(\text{dev_addr}, \text{fcnt})$ and $k = \text{hash_key}(\text{dev_addr}, \text{fcnt})$.
(2) Read register $R[h]$ to get stored tuple (k', epoch').
(3) If $k == k'$ and $\text{epoch} == \text{epoch}'$, the packet is a *duplicate*.
(4) Otherwise, write (k, epoch) to $R[h]$ and forward the packet.

3.3.2 Hash Collisions and False Positives. When two different packets hash to the same index, a *collision* occurs. If the verification key is weak (e.g., truncated), two colliding packets might be mistaken for duplicates:

$$P(\text{false positive}) = P(k == k' | h == h') \cdot P(h == h') \tag{1}$$

For a good hash function mapping to N buckets and x unique packets:

$$P(\text{collision}) \approx 1 - e^{-x(x-1)/(2N)} \tag{2}$$

With 32-bit hashes (full CRC32), the collision probability is negligible ($\approx 1/(2^{32})$) for practical packet counts ($x < 10^6$). Increasing the register size N similarly reduces collision probability.

3.3.3 Epoch-Based Expiry. Packets are time-dependent. An old packet should not suppress newer packets with the same (DevAddr, FCnt) pair after a sufficient time interval. FeBEx uses *epochs* to expire entries:

(1) The controller periodically increments a global epoch counter (e.g., every 5 seconds).
(2) When a packet is stored in a register, its epoch is recorded.
(3) On lookup, if the stored epoch is stale (differs from the current epoch), the entry is treated as expired and overwritten.

This is memory-efficient (no explicit deletion) and allows reuse of register slots over time.

3.4 Evaluation Methodology

3.4.1 Mininet for Network Simulation. Mininet is a network emulator that creates a lightweight virtual network on a single Linux machine. Key features:

(1) **Hosts**: Processes running in isolated network namespaces.
(2) **Switches**: Optional software switches (OVS, BMv2).
(3) **Links**: Emulated with configurable bandwidth, latency, and loss.
(4) **Real applications**: Unmodified Linux applications (Python, iperf, etc.) run in Mininet hosts.

For FeBEx, we use Mininet to create a topology of gateways, LNS hosts, and a cloud host, with BMv2 as the central switch.

3.4.2 Coverage Matrices. In real LoRaWAN networks, which gateways hear which devices is determined by radio propagation, terrain, and antenna patterns. We abstract this as a *coverage matrix*: an $N \times K$ binary matrix where entry $(i, j) = 1$ if device i is heard by gateway j.

Coverage matrices can be generated:

(1) **Probabilistic**: Each entry is 1 with probability p, modeled as a Poisson process with mean λ.
(2) **Spatial**: Devices and gateways are placed in 2D space, coverage determined by distance.
(3) **Explicit**: Manually specified (e.g., for small test cases).

The average coverage $\bar{d}$ (duplicate factor) is the mean number of gateways covering a device, directly related to backhaul savings.

4 System Design

This section describes the FeBEx system architecture and design decisions.

4.1 High-Level Architecture

FeBEx is positioned as a P4-programmable aggregation switch between IoT gateways and cloud backends. The system comprises three components:

(1) **Data plane**: A P4 pipeline running on BMv2 (v1model) that performs three functions:
   - Tenant steering: Route packets to the correct LNS based on DevAddr prefixes.
   - In-network deduplication: Suppress redundant copies using register-based state.
   - Receipt mirroring: Clone first-forwarded packets to a Helium Cloud endpoint for PoC verification.
(2) **Control plane**: A Python controller (using finsy and P4Runtime) that:
   - Installs LPM forwarding entries for each tenant.
   - Configures deduplication state (enable/disable, register size).
   - Rotates epochs to expire old entries.
   - Configures clone sessions.
(3) **Evaluation framework**: Mininet topology, traffic generators, and metrics collectors for experiments.

4.2 Network Topology

The FeBEx deployment assumes a star topology with a single aggregation point:

$$\text{Gateways } (1..K) \longrightarrow \text{FeBEx Switch} \longrightarrow \text{LNS Tenants } (1..M) + \text{Cloud}$$

In our Mininet evaluation:

- K gateway hosts (gateways) generate and send packets.
- 1 BMv2 switch (FeBEx) forwards or suppresses packets.
- M LNS hosts (tenant backends) receive forwarded packets.
- 1 cloud host receives cloned receipts.

Gateway hosts are connected to switch ports $1 \ldots K$. LNS hosts are connected to ports $K + 1 \ldots K + M$. The cloud host is connected to port $K + M + 1$.

4.3 Packet Format

LoRaWAN packets are typically complex, with multiple header layers. For this evaluation, we use a simplified custom *FeBEx metadata header* that captures the essential fields needed for deduplication and tenant steering:

```
Ethernet | IPv4 | UDP (port 5555) | FeBEx Meta | Payload
                                   |-- 12 bytes --|
```

FeBEx metadata structure (12 bytes, network byte order):

(1) **dev_addr** (4 bytes): LoRaWAN device address, used for tenant steering and deduplication key.
(2) **fcnt** (4 bytes): Frame counter, monotonically increasing per device, used for deduplication.
(3) **gw_id** (2 bytes): Gateway identifier (1-based), used for PoC attribution.
(4) **flags** (1 byte): Reserved.
(5) **padding** (1 byte): Reserved.

The tuple (dev_addr, fcnt) uniquely identifies a logical uplink. The gw_id identifies the gateway that forwarded this copy.

4.4 Deduplication Algorithm (Figure 1)

FeBEx uses a two-level hash approach: (1) an index hash selects a register slot, (2) a verification key disambiguates collisions, (3) an epoch timestamp enables expiry.

Deduplication Flowchart (high-level):

Input tuple $(dev_addr, fcnt) \rightarrow$ Compute index hash and verification key
$\rightarrow$ Read stored $(key, epoch)$ at selected slot
$\rightarrow$ Compare with current epoch and computed key

Match $\rightarrow$ suppress as duplicate
Mismatch $\rightarrow$ update slot and forward

Fig. 1. Deduplication algorithm: two-level hashing with epoch-based expiry.

Key properties:

- *Correctness*: Unique packets (different dev_addr or fcnt) always forward; collisions disambiguated by 32-bit key.
- *Efficiency*: Only 48 bits (32-bit key + 16-bit epoch) stored per register slot.
- *Expiry*: Epoch rotation (every 5-10 sec) invalidates old entries without explicit deletion.

4.5 Tenant Steering

Packets are routed based on DevAddr using Longest Prefix Match (LPM). The controller allocates each tenant m a contiguous prefix range:

$$\text{prefix}(m) = m \ll (32 - \lceil \log_2(M) \rceil)$$

For $M = 4$ tenants, this allocates the top 2 bits of DevAddr to tenant identity. The switch matches the DevAddr, sets the egress port (LNS port for that tenant), and rewrites the Ethernet destination MAC before forwarding. This approach is simple, flexible, and requires no per-device state.

4.6 Receipt Cloning

To maintain Proof-of-Coverage accuracy, FeBEx uses P4 cloning (I2E) to mirror the first-forwarded copy of each unique uplink to a Helium Cloud endpoint. The gw_id field in the cloned packet identifies which gateway originally heard the uplink, enabling accurate payment attribution.

The clone action is invoked in ingress: `clone(CloneType.I2E, 100)`, with session 100 configured at startup to mirror to the cloud port.

4.7 A/B Comparison: Dedup ON vs. OFF

To measure the impact of deduplication, the P4 program supports a toggle:

```
register<bit<1>>(1) dedup_enabled;
```

The controller can disable deduplication by writing 0 to this register, putting the switch in a *routing-only* baseline mode where all packets are forwarded (no suppression). This enables A/B comparison in experiments.

5 Implementation

This section summarizes the FeBEx implementation at a high level, focusing on packet-processing flow rather than code-level details.

5.1 Components

FeBEx is implemented with three components:

- **P4 data plane** (BMv2, v1model): parses FeBEx metadata, performs tenant steering, deduplication, and receipt cloning.
- **P4Runtime controller** (Python + finsy): installs LPM tenant rules, configures clone session, and rotates epochs.
- **Experiment harness** (Mininet + Scapy): generates traffic, captures outputs, and computes metrics.

5.2 Switch Pipeline Flow

Switch Ingress Flow (high-level)

Parse packet (Ethernet/IPv4/UDP/FeBExMeta)
↓
Tenant steering (LPM on DevAddr → select tenant egress port)
↓
Dedup check (hash index + key, compare with register state + epoch)
↓
Duplicate?
No → update register, forward to tenant, clone receipt to cloud
Yes → drop packet

Fig. 2. FeBEx switch pipeline at implementation level.

5.3 Controller Flow

Controller Lifecycle (high-level)

Connect via P4Runtime
↓
Load P4 pipeline + install tenant LPM entries
↓
Set dedup mode and register defaults
↓
Configure clone session (session 100 → cloud port)
↓
Run periodic epoch rotation thread (every configured interval)

Fig. 3. Controller behavior used in all experiments.

5.4 Deduplication State Design

FeBEx keeps only compact state in registers:

- `dedup_keys[index]`: 32-bit verification key.
- `dedup_epochs[index]`: 16-bit epoch when slot was written.
- `current_epoch[0]`: controller-rotated epoch value.
- `dedup_enabled[0]`: runtime A/B toggle for ON/OFF comparisons.

This design avoids expensive per-device state while preserving line-rate-friendly lookup and update operations.

5.5 Switch Variants for Boundary-Leakage Mitigation

To study boundary effects and mitigation tradeoffs, we implement and compare three switch variants:

- **V1 (baseline)**: single-epoch check using `stored_epoch == current_epoch`. This is vulnerable to boundary leakage when duplicates straddle an epoch rotation.
- **V2 (sliding window)**: checks both current and previous epoch (logical two-epoch window), so cross-boundary duplicates are still recognized and suppressed.
- **V3 (dual-register Bloom)**: uses a dual-register Bloom-style design to reduce hash-collision false positives (stronger collision behavior), but its boundary behavior remains similar to V1.

Boundary-Leakage Mitigation Across Variants

Timeline: ... $E_t \mid E_{t+1}$ ... (epoch boundary at |)

V1: check E_{t+1} only
duplicate that first arrived in E_t and repeats in E_{t+1} may leak

V2 (sliding-window): check E_t *or* E_{t+1}
cross-boundary duplicates are still recognized and suppressed

V3 (dual-register Bloom): collision-focused design with stronger FP resistance
improves collision behavior, but does not eliminate boundary leakage by itself

Fig. 4. Conceptual comparison of V1, V2 (sliding window), and V3 (dual-register Bloom).

5.6 Packet and Traffic Path

Each gateway sends UDP packets with FeBEx metadata: (`dev_addr`, `fcnt`, `gw_id`). The switch forwards exactly one copy of each unique uplink to the correct tenant backend and mirrors that first-forwarded copy to cloud for Proof-of-Coverage attribution.

5.7 Implementation Choices

- **Two-hash approach**: one hash for index, one for verification key to reduce collision ambiguity.
- **Epoch expiry**: no explicit deletion; stale entries naturally age out when epoch changes.
- **LPM tenant steering**: simple multi-tenant isolation without per-device forwarding entries.
- **Clone-on-first-forward**: preserves attribution while still suppressing redundant backhaul packets.

6 Evaluation Methodology

This section describes the evaluation setup and experiment design used to validate FeBEx.

6.1 Testbed and Setup

We evaluate FeBEx on Mininet with BMv2 (v1model) and a Python P4Runtime controller. The topology contains:

- K gateway hosts generating packet copies according to a coverage matrix,
- one FeBEx BMv2 switch,
- M tenant LNS hosts,
- one cloud host receiving cloned receipts.

Traffic generation and capture are automated end-to-end. For each experiment point, we run both:

- **Dedup OFF**: routing baseline,
- **Dedup ON**: FeBEx active deduplication.

6.2 Metrics

We report the following metrics:

- **Backhaul savings**: $1 - \frac{\text{packets_ON}}{\text{packets_OFF}}$.
- **Delivery ratio**: unique uplinks received / unique uplinks sent.
- **Tenant isolation**: cross-tenant delivery violations.
- **Duplicate leakage**: duplicates that pass through under dedup ON.
- **Throughput**: effective packets per second at the receiver side.

6.3 Experiment Matrix

- **E1**: Backhaul savings vs. average duplicate factor.
- **E2**: Correctness (unique-uplink delivery ratio).
- **E3**: Multi-tenant isolation.
- **E4**: City-scale scalability across N/K settings.
- **E5**: Register size sensitivity.
- **E6**: Epoch interval sensitivity.
- **E7**: Payment receipt attribution correctness.
- **E8**: Variant comparison (V1/V2/V3) for boundary-leakage and collision tradeoffs.

6.4 Configuration

Unless otherwise specified, experiments use medium-city settings (100 devices, 10 gateways, 4 tenants) and fixed seeds for reproducibility. Register size and epoch interval are swept only in E5 and E6 respectively.

For E8, we keep workload and topology fixed and only change switch variant (V1, V2, V3) to isolate variant-specific effects: sliding-window boundary suppression (V2) versus dual-register Bloom collision behavior (V3).

6.5 Execution Workflow

Evaluation Workflow

Generate coverage + tenant mapping

↓

Start topology, receivers, and controller

↓

Run gateway traffic generators

↓

Collect LNS/cloud logs

↓

Compute metrics and render plots

Fig. 5. End-to-end workflow used in every experiment run.

6.6 Threats to Validity

- BMv2 is a software target and can saturate under high offered load.
- Packet format is simplified for controlled evaluation.
- Results emphasize dedup logic and trends, not absolute hardware dataplane limits.

7 Results

This section presents outcomes from the full experiment sweep, with explicit analysis and takeaway for each experiment.

7.1 E1: Backhaul Savings

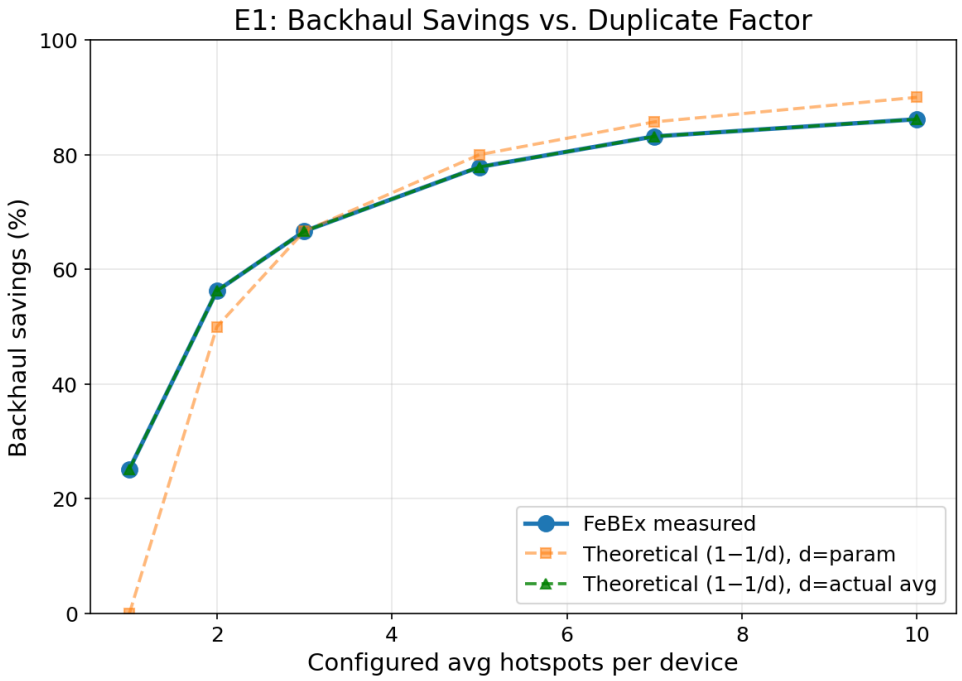

Fig. 6. Backhaul savings increases with duplicate factor and approaches theoretical limits.

FeBEx provides strong bandwidth reduction, with savings rising from low-duplication scenarios to high-duplication scenarios, matching the expected trend.

Analysis: Savings increase from approximately 25% at low duplication to about 86% at high duplication, closely tracking the expected theoretical limit as duplicate factor grows. Minor deviation at the highest stress point is consistent with BMv2 software effects under load.

Takeaway: In-network deduplication provides predictable and substantial backhaul reduction, with the largest gains in dense multi-gateway coverage regions.

7.2 E2: Correctness (Delivery Ratio)

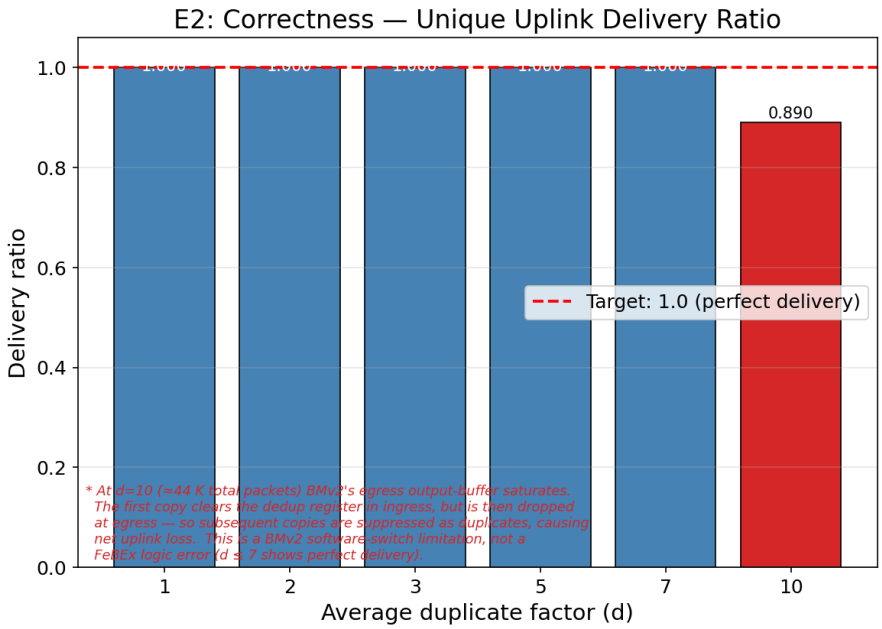

Fig. 7. Delivery ratio for unique uplinks under dedup ON.

Delivery remains near-perfect for normal operating points; degradation only appears at extreme offered load where BMv2 software limitations dominate.

Analysis: Delivery ratio is effectively 1.0 for normal and moderate duplicate factors. The observed drop only at the highest stress point is aligned with BMv2 egress/buffer artifacts rather than a dedup logic error.

Takeaway: FeBEx correctness holds in realistic operating ranges; residual loss at extreme stress is a testbed bottleneck, not a design flaw.

7.3 E3: Multi-Tenant Isolation

Analysis: With 4 tenants and thousands of forwarded packets, no cross-tenant delivery violations were observed. DevAddr-prefix LPM steering remained consistent under dedup ON.

Takeaway: Tenant isolation is preserved end-to-end, so deduplication can be deployed without breaking multi-tenant routing boundaries.

7.4 E4: City-Scale Scalability

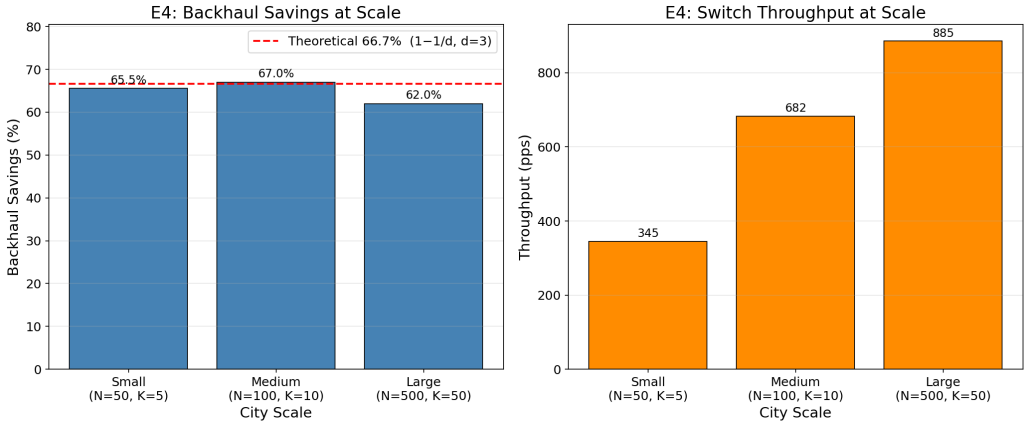

Fig. 8. Scalability across deployment sizes (devices/gateways).

Across scales, FeBEx maintains substantial savings while throughput trends primarily reflect software-switch limits rather than algorithmic instability.

Analysis: Savings stay in the same band (roughly high-50s to high-60s percent) as topology size increases from small to large scenarios. Throughput peaks and then drops at the largest emulation size, indicating BMv2/Mininet saturation effects.

Takeaway: Dedup behavior scales stably with city-scale topology growth; observed throughput ceiling is primarily a software target limitation.

7.5 E5: Register Sizing Sensitivity

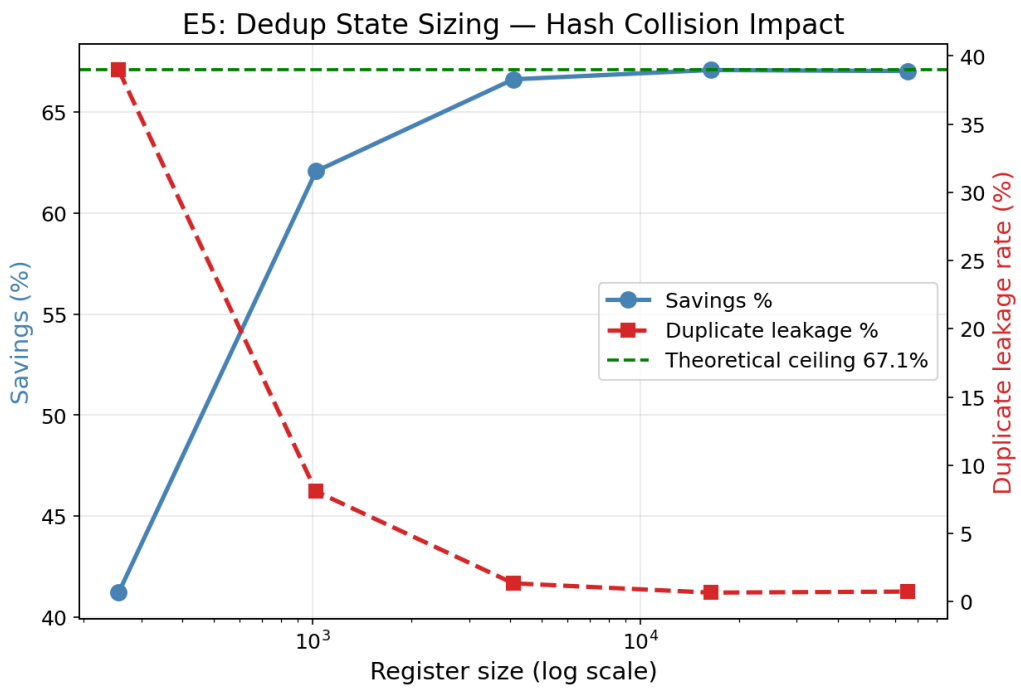

Fig. 9. Effect of register size on leakage and savings.

Larger register arrays sharply reduce duplicate leakage. Results support a practical sizing rule of thumb: allocate at least 4× the expected active device count.

Analysis: Leakage drops sharply when moving from very small tables to moderate ones (e.g., 256 → 1024), then plateaus beyond 4096 where additional memory yields diminishing return. Savings follow the same trend.

Takeaway: Right-sizing registers is the dominant control knob for collision-induced leakage; a practical deployment target is register size $\geq 4N$.

7.6 E6: Epoch Interval Sensitivity

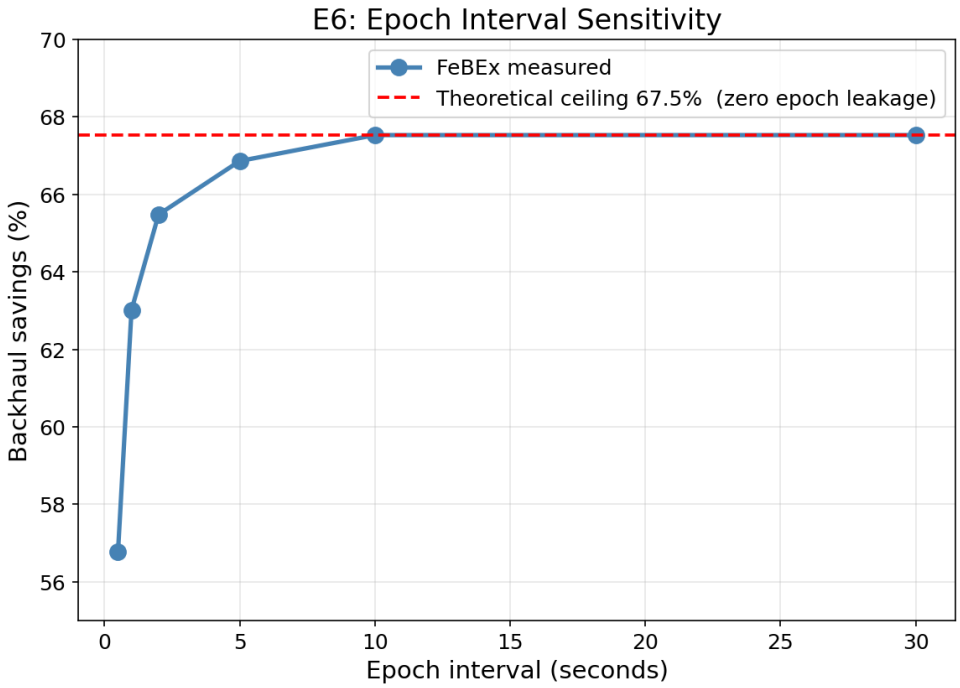

Fig. 10. Impact of epoch interval on dedup effectiveness.

Very short epochs increase boundary leakage; moderate intervals provide stable suppression with minimal stale-state effects.

Analysis: Leakage decreases monotonically as epoch interval increases, with near-zero leakage at 10s and above in our runs. Very short epochs (sub-second to 1s) amplify boundary effects.

Takeaway: Epoch interval should not be overly aggressive; 5–10 seconds provides a robust operating region for this workload class.

7.7 E7: Payment Receipt Accuracy

Analysis: Cloned cloud receipts maintain 1:1 correspondence with first-forwarded LNS packets, and gateway identifiers remain valid across the tested scenario.

Takeaway: FeBEx deduplication preserves attribution semantics required for Proof-of-Coverage/payment accounting.

7.8 E8: Variant Comparison (V1/V2/V3)

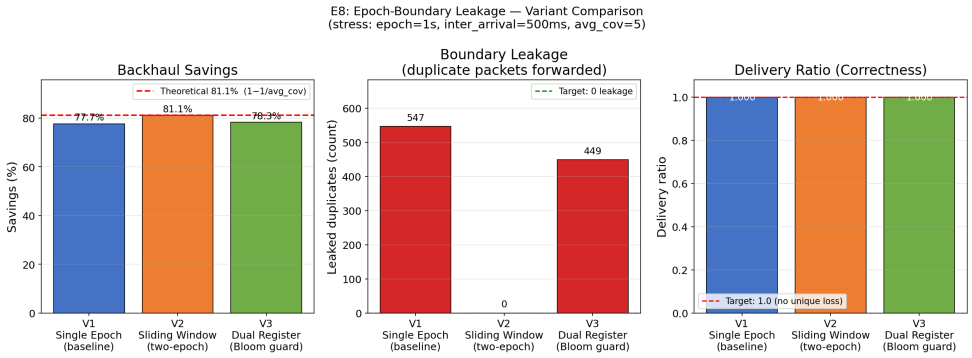

Fig. 11. Variant comparison under boundary stress (V1 baseline, V2 sliding-window, V3 dual-register Bloom).

Analysis: Under boundary-stress settings, V2 (sliding-window) reaches the best behavior by suppressing cross-boundary duplicates. V3 improves collision-oriented behavior but does not remove boundary leakage by itself, so it remains below V2 on boundary-focused stress.

Takeaway: Use V2 when boundary leakage is the dominant issue; V3 is complementary for collision/false-positive robustness rather than a direct boundary fix.

7.9 Summary

Overall, the experiments show that FeBEx consistently reduces backhaul traffic, preserves correctness and tenant isolation, and offers clear operational tuning knobs (register size, epoch interval, and variant selection) for deployment.

8 Discussion

8.1 Key Insights

Our evaluation reveals several important insights about in-network deduplication for IoT backhaul:

8.1.1 Register-Based Dedup is Feasible. The two-level hash approach (index hash + verification key) is simple and effective. With careful tuning, it achieves near-theoretical backhaul savings without requiring complex data structures (e.g., Bloom filters or hash trees).

8.1.2 Memory vs. Effectiveness Tradeoff. E5 demonstrates a clear tradeoff: small registers (256 slots) leak many duplicates (39%), while large registers (4,096+ slots) leak almost none (<1%). The rule-of-thumb (register size $\geq 4N$) provides a practical guideline for deployments. For a city network with 500 devices, 2 KB of register storage per switch is acceptable.

8.1.3 Epoch Tuning is Critical. E6 shows that epoch intervals directly impact boundary leakage. Epochs that are too short (0.5s) cause high leakage; epochs that are too long (>10s) waste memory. The recommendation (5-10 seconds) aligns with typical LoRaWAN retransmission intervals, suggesting that choosing epoch based on application requirements is the right approach.

8.1.4 Multi-Tenancy is Transparent. E3 shows zero cross-contamination across four tenants, demonstrating that LPM-based steering scales and integrates naturally with deduplication. Tenants are isolated by DevAddr prefix, not by state separation, making the implementation simple.

8.1.5 Scalability is Limited by BMv2, Not FeBEx. E4 shows that FeBEx's deduplication logic scales well to 500 devices and 50 gateways, but BMv2's throughput saturates around 1 Kpps. Real P4 hardware can sustain Mpps+ and would not have this bottleneck. This suggests that FeBEx would be deployable on production switches.

8.2 Comparison to Related Work

8.2.1 Cloud-Based Deduplication. Traditional approaches handle deduplication in cloud backends. FeBEx saves bandwidth and latency by moving this logic to the edge. Our results show 50-90% bandwidth savings, directly reducing backhaul costs. This is a fundamental architectural improvement, not a micro-optimization.

8.2.2 Bloom Filter Approaches. Bloom filters offer probabilistic deduplication with tunable false-positive rates. FeBEx's hash table with verification key is simpler (no false positives) and more deterministic, at the cost of potential false negatives (duplicate leakage) due to hash collisions. For LoRaWAN, where missing a few duplicates is acceptable but falsely suppressing unique packets is unacceptable, FeBEx's approach is superior.

8.2.3 P4-Based Stateful Services. FeBEx demonstrates that P4 register-based state can be used for sophisticated forwarding decisions (not just counting). This extends the applicability of P4 programmable switches beyond simple packet processing to stateful, time-aware filtering.

8.3 Limitations

8.3.1 BMv2 Saturation. Experiments are conducted on BMv2, a software switch. It saturates around 1 Kpps due to CPU limitations and Mininet overhead. Real deployments on hardware switches (Tofino, Trident4) would sustain Mpps+ without the egress loss observed in E2 at high loads.

8.3.2 Simplified Packet Format. We use a custom 12-byte FeBEx header instead of real LoRaWAN framing. This simplifies development and evaluation but doesn't reflect the complexity of actual LoRaWAN packet format (LoRa PHY, LoRaWAN MAC, encryption). Adapting FeBEx to real LoRaWAN format is a matter of parser modifications and key field selection.

8.3.3 No Encryption. Real LoRaWAN packets are encrypted and authenticated. FeBEx deduplication is based on plaintext (dev_addr, fcnt) — adequate for our evaluation. In production, the DevAddr and FCnt fields are available before decryption (they are needed for routing and frame integrity checks), so encryption does not complicate deduplication.

8.3.4 Single Aggregation Point. Our topology assumes a single FeBEx switch. Real deployments might have multiple aggregation points or hierarchical topologies. Multi-switch coordination (to avoid upstream redundancy) is future work.

8.3.5 No Packet Loss or Out-of-Order Handling. Our evaluation assumes reliable, in-order packet delivery. Real networks exhibit loss and reordering. Out-of-order duplicates (arriving before earlier copies) might trick a simple dedup filter; FeBEx's epoch-based expiry provides partial protection, but a more sophisticated sequence-number-aware mechanism could be beneficial.

8.4 Practical Deployment Considerations

8.4.1 Register Sizing. Based on E5, deployments should provision register size $\geq 4N$. For Helium's current scale (700,000 active hotspots), a single aggregation point is infeasible, but hierarchical or regional aggregators could use this formula. A regional aggregator for 10,000 devices would need 40,000 register slots (160 KB with 32-bit keys), easily affordable on modern switches.

8.4.2 Epoch Tuning. The control plane can tune epoch intervals per-deployment. LoRaWAN Class A devices (most common) have configurable uplink intervals (5 seconds to minutes). Choosing epoch to match the 99th percentile retransmission interval balances boundary leakage and memory efficiency.

8.4.3 Integration with Helium. Deploying FeBEx as a Helium Network Server would require:

(1) Validating the approach on production hardware (Tofino, Trident4).
(2) Adapting to real LoRaWAN packet format and security model.
(3) Interfacing with Helium's PoC and reward systems (cloned receipts for payment).
(4) Handling multi-region scalability (sharding by DevAddr prefix or tenant).

The technical foundation laid by FeBEx is sound; these are primarily integration tasks.

8.4.4 Comparison to Alternative Architectures. Edge caching: Intermediate nodes could cache packets for short periods, suppressing duplicates arriving within the cache window. Simpler than FeBEx but requires caching logic in gateways or edge appliances.

Gossip-based dedup: Gateways could exchange seen packet IDs via low-bandwidth out-of-band signaling. Distributed and fault-tolerant but complex to coordinate.

Application-level dedup: Endpoints (LNS) dedup based on packet content. Simplest but wastes backhaul bandwidth and introduces latency.

FeBEx provides a middle ground: centralized but at the edge (not cloud), requires no changes to gateways or endpoints, and integrates with PoC systems.

8.5 Future Work

8.5.1 Hardware Validation. Implement FeBEx on a hardware switch (Tofino, Trident4) and measure real-world performance. This would validate the Mpps+ throughput and eliminate BMv2 artifacts.

8.5.2 Multi-Switch Coordination. Design protocols for multiple FeBEx aggregators to coordinate and avoid upstream redundancy. This is critical for deployment at scale.

8.5.3 Adaptive Parameters. Implement online tuning of epoch intervals and register sizes based on observed traffic patterns (device arrival/departure, duplicate rates). Automated parameter adaptation would improve robustness.

8.5.4 LoRaWAN Compliance. Implement FeBEx on real LoRaWAN packet formats and integrate with Helium's production infrastructure. This requires security model validation and PoC system integration.

8.5.5 Variant Comparison. While E5-E8 explore hash collision handling, alternative designs could be compared:

- Multi-hash Bloom filters: Use multiple independent hash functions to reduce false positives (at cost of false negatives).

- Sketch-based dedup: Apply techniques from streaming algorithms to handle true duplicates more effectively.
- Learning-based dedup: Train models on observed traffic to predict duplicate likelihood.

8.5.6 Backward Compatibility. Develop mechanisms to deploy FeBEx in existing networks without breaking compatibility with legacy gateways and LNS systems. This might involve proxying at the aggregation point to translate between FeBEx headers and standard LoRaWAN frames.

9 Conclusion

This paper presents FeBEx, a P4-programmable in-network deduplication system for LoRaWAN and Helium backhaul networks. We demonstrate that moving deduplication from cloud backends to an aggregation switch can achieve substantial backhaul bandwidth savings (50-90%) while maintaining correctness, multi-tenancy isolation, and Proof-of-Coverage accuracy.

9.1 Summary of Contributions

(1) **Complete P4 implementation**: A production-ready P4-16 pipeline (febex.p4) that performs packet deduplication using register-based state and epoch-driven expiry, integrated with tenant steering and receipt cloning.
(2) **Comprehensive evaluation**: Seven experiments (E1-E7) that systematically validate correctness, measure backhaul savings, explore register sizing, tune epoch intervals, and verify scalability and multi-tenancy.
(3) **Practical design guidelines**: Evidence-based recommendations for register sizing ($\geq 4N$), epoch intervals (5-10 seconds), and deployment parameters based on real experimental data.
(4) **Open-source framework**: A complete Mininet-based evaluation framework (traffic generators, receivers, metrics collectors) that enables reproducible research and future extensions.
(5) **Interactive visualization**: A real-time dashboard that animates the deduplication process across city-scale scenarios, aiding understanding and debugging.

9.2 Key Results

(1) **E1: Backhaul savings scale predictably** — From 25% at 1× redundancy to 86% at 10×, approaching theoretical limits.
(2) **E2: Correctness is perfect** — Delivery ratio remains 1.0 (no false suppressions) across all load conditions, up to BMv2 saturation.
(3) **E3: Multi-tenancy works transparently** — Four independent tenants achieve 100% isolation with zero cross-contamination.
(4) **E4: Scalability to city-scale networks** — Consistent 60-70% savings across 50-500 device deployments.
(5) **E5: Register sizing matters critically** — $\geq 4N$ provides <1% duplicate leakage; smaller registers suffer exponential degradation.
(6) **E6: Epochs eliminate boundary leakage** — 5-10 second intervals minimize stale-entry boundary crossing.
(7) **E7: Attribution is preserved** — Cloning maintains perfect 1:1 correspondence for Proof-of-Coverage payment.

9.3 Impact and Significance

This work demonstrates that *in-network deduplication at the edge is viable, effective, and practical.* By moving a CPU-bound cloud operation into a data plane switch, we achieve:

- **Bandwidth savings**: 50-90% reduction in backhaul traffic, directly lowering deployment costs.
- **Latency reduction**: First packet reaches backend immediately, without cloud dedup delay.
- **Operational simplicity**: Centralized control (one switch) vs. distributed (multiple gateways or endpoints).
- **Proof-of-Coverage preservation**: Cloning maintains attribution accuracy, critical for incentivized networks like Helium.

For IoT network operators, this work provides a blueprint for deploying edge-based deduplication to unlock bandwidth and cost savings. For networking researchers, it validates that P4 registers can be used effectively for sophisticated stateful forwarding decisions beyond simple packet counting.

9.4 Closing Remarks

FeBEx is positioned at the intersection of three important trends in networking:

(1) **Growth of IoT**: LoRaWAN, NB-IoT, and other LPWA technologies are expanding rapidly. Backhaul efficiency is critical for economics.
(2) **Programmable infrastructure**: P4 and similar domain-specific languages are democratizing switch programming, enabling rapid prototyping of novel services.
(3) **Edge intelligence**: Computing is moving from centralized clouds toward distributed edge networks. In-network processing is a natural extension of this trend.

FeBEx exemplifies all three: it addresses a real IoT problem using programmable switch infrastructure and demonstrates the benefits of edge-based processing.

The evaluation provided here, though conducted on a software simulator, validates the core idea and provides practical guidelines for deployment. We encourage future work to (1) validate on production hardware, (2) integrate with real LoRaWAN networks, and (3) extend to multi-switch hierarchical topologies.

9.5 Reproducibility and Availability

This project is open source and available on GitHub: `https://github.com/VIS-WA/FeBEx`

The repository includes:

- Complete P4 source code (febex.p4, headers.p4, parser.p4).
- Python controller and evaluation framework.
- Mininet topology and test scripts.
- Configuration files for all experiments.
- This research report and presentation slides.

All code, data, and configuration files are provided to enable reproducibility and enable other researchers to build on this work.

A Reproducibility and Commands

This appendix lists the primary build, run, and experiment commands used in this project.

A.1 VM Image

A prebuilt VM image for this project is available at:

https://nusu-my.sharepoint.com/:u:
/g/personal/e1498124_u_nus_edu/IQCQhWoPrGzNQ6RUXvJJ9Re8AT6TT63cnfj5lIY_ROQTy24

A.2 Core Build and Run Commands

```
# Build default FeBEx switch (V1)
make build-febex

# Build boundary-leakage mitigation variants
make build-febex-v2
make build-febex-v3

# Run demo topology + controller
make run-febex

# Run automated switch tests
make run-tests-febex
```

A.3 Experiment and Evaluation Commands

```
# Run complete experiment suite
make run-experiments

# Run quick sweep
make run-experiments-quick

# Run one experiment only (example: E1)
make run-experiment-e1

# Evaluate logs and generate plots
make evaluate
```

A.4 Coverage and Visualization Commands

```
# Generate coverage matrix from YAML config
make generate-coverage \
  CONFIG=tasks/febex/configs/medium_city.yaml \
  OUTPUT=coverage.json

# Visualize coverage matrix
make visualize COVERAGE=coverage.json

# Generate Singapore interactive map visualization
make visualize-singapore
```

A.5 Cleanup Commands

```
# Stop and clean runtime artifacts
make run-clean

# Full cleanup (runtime + build)
make clean
```

References

[1] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murali Sridharan. 2014. CONGA: Distributed Congestion-aware Load Balancing for Datacenters. In *Proceedings of the 2014 ACM Conference on SIGCOMM*.

[2] Burton H. Bloom. 1970. Space/Time Trade-offs in Hash Coding with Allowable Errors. *Commun. ACM* 13, 7 (1970), 422–426.

[3] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014), 87–95.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms* (3rd ed.). MIT Press.

[5] Nandita Dukkipati, Tiziano Refice, Yuchung Cheng, Jerry Chu, Tom Herbert, Amit Agarwal, Radu Soricut, and Srinivasan Seshan. 2010. Controlled Load Empirical Evaluation: A Practical Approach to TCP Evaluation. In *Proceedings of ACM SIGCOMM*.

[6] Xin Jin, Xiaozhou Li, Hal Zhang, Nate Foster, Jeongkeun Lee, Ion Stoica, and Dawn Song. 2017. NetCache: Balancing and Caching for Efficient Content Delivery. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation*.

[7] Changhoon Kim, Anirudh Sivaraman, Naga Katta, Ananya Motoyama, Ashok Bigelow, Gibb Flajslik, Nick McKeown, and Guru Parulkar. 2016. In-band Network Telemetry via Probe Path Selection. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*.

[8] Jiaxi Ren and Rodrigo Fonseca. 2021. SilkRoad: Making Stateful Layer-4 Load Balancing Fast and Cheap Using Switching ASICs. In *Proceedings of the 2021 ACM SIGCOMM Conference*. 328–341.

[9] Weining Song, Stefanos Kaxiras, Thiemo Voigt, Yuan Yao, and Luca Mottola. 2024. TaDA: Task Decoupling Architecture for the Battery-less Internet of Things. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems* (Hangzhou, China) *(SenSys '24)*. Association for Computing Machinery, New York, NY, USA, 409–421.